

## Тестовое задание для джуниор-разработчика

### Общая цель

Создать веб-приложение для управления проектами и загрузки изображений — как клиентскую часть, так и (опционально) серверную. Весь стек должен быть упакован в Docker. Если используются несколько сервисов — сборка осуществляется через docker-compose.

### Часть 1: Frontend

#### Задача 1 (полегче) : Управление проектами и изображениями

Реализуйте веб-интерфейс со следующими возможностями:

- Отображение списка проектов.
- Создание нового проекта по названию.
- Переход внутрь проекта и просмотр списка загруженных изображений.
- Загрузка одного или нескольких изображений в проект (через кнопку или drag-and-drop).
- Предварительный просмотр изображений в виде миниатюр.
- Удаление проекта и отдельных изображений.

Хранение данных — локальное (например, localStorage). Бэкенд не требуется. Интерфейс должен быть адаптивным и собираться в Docker-образ (например, на основе nginx или статического сервера).

#### Задача 2 (посложнее): Интерактивная аннотация изображений

На отдельной странице (или внутри просмотра изображения из проекта) реализуйте:

- Отображение выбранного изображения.
- Возможность вручную рисовать bounding box'ы (прямоугольники) на изображении — например, с зажатой мышью.
- После завершения рисования — открывать форму для назначения класса этому bounding box'у (например, выпадающий список или текстовое поле).
- Каждый bounding box должен храниться как отдельный объект с координатами, классом и уникальным идентификатором.
- При наведении курсора на bounding box — отображать всплывающую подсказку с информацией: класс, координаты, размеры.
- Возможность редактировать или удалять уже созданные bounding box'ы (например, кликом по ним или через контекстное меню).

Данные аннотаций могут храниться локально (в памяти или localStorage) — синхронизация с бэкендом не требуется на этом этапе.

Дополнительно приветствуется:

- Имитация вызова ML-эндпоинта (POST /predict) с возвратом фейковых bounding box'ов.
- Поддержка масштабирования изображения и корректная работа аннотаций при зуме.

Технологии — на выбор: React, Vue, Svelte или чистый JavaScript. Весь фронтенд должен собираться в Docker-образ.

## Часть 2: Fullstack

Реализуйте полноценное веб-приложение с бэкендом на FastAPI и базой данных.

Бэкенд должен предоставлять следующие эндпоинты:

- POST /projects — создать проект с указанным названием.
- GET /projects — получить список всех проектов.
- DELETE /projects/{id} — удалить проект и все связанные с ним изображения.
- POST /projects/{id}/images — загрузить одно или несколько изображений в проект.
- GET /projects/{id}/images — получить метаданные и ссылки на изображения проекта.
- DELETE /images/{image\_id} — удалить конкретное изображение.

Требования к бэкенду:

- Использовать FastAPI в связке с SQLAlchemy (или другим ORM).
- Хранить метаданные проектов и изображений в PostgreSQL или SQLite (PostgreSQL предпочтительнее).
- Продумать архитектуру хранения изображений. Локальное хранение допустимо на этапе тестового задания, но решение должно позволять легко перейти к внешнему хранилищу (например, к S3-совместимому бакету). Укажите в README, как будет реализована такая замена.
- Загруженные изображения должны быть доступны по публичному URL.
- Все сервисы (бэкенд, база данных, фронтенд) должны запускаться через docker-compose. Использование MinIO или другого S3-совместимого хранилища — необязательно, но крайне рекомендуется.

Дополнительно приветствуется:

- Эндпоинт POST /predict/{image\_id}, имитирующий вывод ML-модели в формате COCO-like аннотаций (bounding box + класс).
- Эндпоинт POST /images/{image\_id}/annotations и GET /images/{image\_id}/annotations — для сохранения и получения аннотаций, созданных пользователем (bounding box'ы + классы).

Требования к сборке и сдаче

Весь проект должен быть представлен в виде одного репозитория на GitHub.

В корне репозитория должен находиться файл docker-compose.yml, запускающий всё приложение одной командой (docker-compose up --build).

Должны быть инструкции в README.md:

- Как запустить проект.
- Какие технологии использованы.